From Researcher to Orchestrator: Attention-Bounded Supervision of AI Agent Teams in Engineering Research

Dan McCreary
Independent Researcher, Minnesota, USA
https://www.linkedin.com/in/danmccreary/
ORCID: 0009-0009-3542-8828

Version 0.01
October 22, 2026

Abstract

Researchers in electrical and computer engineering increasingly delegate literature search, coding, data analysis and drafting to AI agents, yet practices for supervising several agents at once lag behind adoption, and the researcher remains accountable for every claim. Existing multi-agent frameworks and levels-of-automation models do not tell a researcher how much autonomy to grant a task, how work should pass between agents, or how many agents one person can supervise. This paper presents an orchestration framework that answers these questions. Each task's autonomy level is set from three properties of its effects (reversibility, reach and verifiability) by a rule that is complete and monotone. Work passes between agents as version-controlled files that no agent may mark as accepted when it produced them, and every reported number must name the computation that produced it. A review-load model bounds team size by the researcher's daily attention and shows that the ratio of planning to execution agents follows from their review costs rather than being fixed. We apply the framework to an illustrative analog-to-digital converter characterization task and to two AI-assisted publication projects, a 20-chapter textbook and an arXiv paper, whose logs and repositories record 13 defects. Mechanical checks caught every defect for which one existed, errors of meaning were caught only when someone recomputed or read the work, and a quality score that no computation had produced escaped every control and was published. The results suggest measuring review load and escaped defects before enlarging an agent team.

Keywords: AI agents, human supervisory control, levels of autonomy, multi-agent systems, research workflows, verification, reproducibility, electrical engineering

1 Introduction

Large language model (LLM) agents now carry out multi-step research tasks rather than answering single questions. An agent interleaves reasoning with tool calls—searching, reading files, running code—and observes the results before choosing its next action [1, 2, 3]. Researchers have adopted these tools quickly. In a 2025 survey of 2,430 researchers, 84% reported using AI tools, up from 57% a year earlier [4]; earlier surveys and corpus studies found the same trend in attitudes and in the text of published papers [5, 6]. Electrical and computer engineering is no exception: LLMs already generate and debug register-transfer-level hardware descriptions [7, 8, 9], and domain-adapted models assist chip-design engineers [10].

Practice has not kept pace with adoption. In the same survey, 57% of researchers named a lack of guidelines and training as a barrier, and the share concerned about inaccurate output rose from 51% to 64% [4]. The concern is well founded. Language models produce fluent but false statements [11]; in one study, 55% of the bibliographic citations produced by GPT-3.5 and 18% of those produced by GPT-4 were fabricated [12]. Models tend to agree with their users [13], coding agents can satisfy tests by exploiting them instead of meeting the specification [14], and an analysis of more than 1,600 execution traces from seven multi-agent frameworks identified 14 distinct failure modes [15]. Meanwhile, venues and funders agree that an AI tool cannot be an author and that the human authors take full responsibility for everything a paper contains [16, 17, 18, 19]. The researcher therefore remains accountable for work that is increasingly performed by systems that act on their own.

Two bodies of work bracket this problem without solving it. Multi-agent frameworks and "AI scientist" systems coordinate agents with each other and push toward greater autonomy [20, 21, 22, 23, 24]; the human, where present, is an approver of last resort, and work passes between agents as conversation. Research on levels of automation and human supervisory control models exactly the quantity that matters—how much a human operator can oversee—but it was developed for operators of vehicles and robots, not for agents that deliver asynchronous documents and code whose cost lies in verification [25, 26, 27, 28]. Recent proposals grade the autonomy of AI agents by the role the user plays [29], but they do not say how to assign a level to a given task or how many agents a single person can responsibly supervise. What is missing is an operational framework that tells a researcher three things: how much autonomy to grant each task, how work should pass between agents so that it stays reviewable, and how large a team their attention can support.

This paper develops such a framework and examines it against three cases. It condenses and formalizes the central argument of the author's open textbook *AI for EE Research* [30]: that the researcher's role shifts from performing each task to *orchestrating* a team of planning and execution agents, while remaining accountable for what is claimed. Our thesis is that the binding constraint on such a team is not the number of agents that can be launched but the researcher's review attention, and that a framework which makes this constraint explicit also places checks where the defects that matter occur.

This paper makes the following contributions:

1. **An orchestration framework for research agent teams** (Section 3): defined roles, a four-level autonomy scale, and an assignment rule that sets each task's level from three properties of its effects—reversibility, reach and verifiability. We show that the rule is complete and monotone.

2. **A file-based handoff protocol** with five statuses and a separation-of-acceptance invariant, under which no agent may accept its own output, together with a provenance rule for reported numbers that the case analysis showed to be necessary.

3. **An attention-budget model of review load** that yields the span of control of one researcher, shows that the ratio of planning to execution agents follows from their review costs rather than being a constant, and relates both to the fan-out models of human supervisory control.

4. **A three-case evaluation**: an illustrative characterization task from analog-to-digital converter (ADC) research, and two real AI-assisted publication projects—a 20-chapter textbook and an arXiv paper—whose logs and repositories record 13 defects, analyzed by the control that caught each one or failed to.

Section 2 reviews related work on agents, automated science, LLMs in hardware design, supervisory control and verification. Section 3 presents the framework. Section 4 states the research ques-

tions and the case-study method and reports the three cases and the cross-case analysis. Section 5 discusses implications and limitations, and Section 6 concludes with directions for measurement.

2 Background and Related Work

Five strands of prior work bear on how a researcher should supervise a team of agents. Each subsection summarizes one strand and states how this paper builds on it or departs from it. Table 1 summarizes the comparison.

2.1 LLM agents and multi-agent frameworks

An LLM agent couples a language model with tools and a loop: it decides on an action, calls a tool, observes the result and decides again [1]. Surveys organize the design space of single agents, for example into profile, memory, planning and action modules [2], and of societies of cooperating agents [3]. Multi-agent frameworks compose several such agents. AutoGen builds applications from conversable agents that exchange messages [20]; MetaGPT encodes standardized operating procedures and assigns software-company roles to agents [21]; ChatDev organizes role-playing agents into chat-based development phases [22]; CAMEL studies cooperation between role-playing agents [31]; and HuggingGPT uses one model as a controller that plans tasks and dispatches them to specialist models [32]. Separating planning from execution also helps at the level of a single prompt [33]. Practitioner reports distinguish *workflows*, whose steps are fixed in code, from *agents*, which direct their own tool use, and recommend an orchestrator–worker pattern for tasks that cannot be decomposed in advance [34]; a production research system built this way used roughly fifteen times the tokens of a chat interaction [35]. Agent behavior is increasingly configured through plain files—project instruction files such as `AGENTS.md` [36] and packaged *skills* with a `SKILL.md` entry point [37]—and coding agents are evaluated on real repository tasks [38]. The failures of multi-agent systems cluster into system-design issues, inter-agent misalignment and inadequate task verification [15].

This paper adopts the planner–executor and orchestrator–worker patterns and the use of plain files as configuration. It departs from the frameworks above in two respects. The orchestrator is the human researcher, whose review time is treated as the scarce resource, and work passes between agents as versioned files with an explicit status, not as conversation, so that every handoff can be read, diffed and accepted or rejected.

2.2 AI systems that automate science

A parallel line of work aims to automate research itself. The AI Scientist generates ideas, runs experiments, writes papers and reviews them automatically [23]. Coscientist designs, plans and executes chemistry experiments through laboratory automation [39], and ChemCrow augments a model with expert chemistry tools [40]. Agent Laboratory takes a human-provided idea through literature review, experimentation and report writing [24], and Co-Scientist uses a multi-agent system to generate and refine hypotheses [41]. Critics warn that such tools can produce illusions of understanding, in which scientists believe they understand more than they do, and can narrow the questions a field asks [42]. Others argue that risk to people grows with an agent's autonomy and that fully autonomous agents should not be built [43].

Our framework sits at the opposite end of the autonomy range. It keeps the framing of the question, the choice between approaches and the acceptance of every reported claim with the researcher, consistent with the accountability required by venues and funders [16, 17, 18, 19]. It

also starts from a constraint specific to engineering: much of the evidence is a physical measurement, which an agent can analyze but must never be allowed to stand in for.

2.3 LLMs in hardware design

In electrical and computer engineering, work on LLMs has concentrated on design artifacts. Benchmarks measure the quality of generated Verilog [7, 9]; conversational design studies show an engineer and a model co-designing hardware through dialogue [8]; and domain-adapted models support engineering assistance, script generation and bug summarization in chip design [10]. These studies ask how capable a model is on a design task. This paper asks a complementary question: given capable models, how should the researcher who uses them divide, supervise and verify the work of several at once.

2.4 Levels of automation and human supervisory control

Sheridan and Verplank's ten-level scale for supervisory control of teleoperators is the origin of most *levels of automation* [25]. Parasuraman, Sheridan and Wickens separated the type of function automated (information acquisition, analysis, decision and action) from the level of automation, and proposed that the level be chosen by evaluating human-performance consequences together with automation reliability and the costs of decision and action outcomes [26]. Mixed-initiative interfaces weigh the expected value of acting autonomously against that of asking the user [44]; design guidelines for human–AI interaction [45] and human-centered AI [46] argue that high automation and high human control can coexist. For AI agents, Feng, McDonald and Zhang define five levels by the role the user plays—operator, collaborator, consultant, approver and observer—and propose autonomy certificates; they treat autonomy as a design decision separate from capability [29]. Visibility measures such as agent identifiers, real-time monitoring and activity logs make agents' actions inspectable after the fact [47]. Our own earlier work described advocate agents that act on a person's behalf using explicit, machine-readable preferences [48].

A second tradition asks how many semi-autonomous systems one person can supervise. Crandall et al. characterized each robot by its *neglect tolerance*—how long it can be left unattended before its performance degrades—and by the interaction time needed to restore it, and used the two to estimate the maximum number of robots one person can manage, a quantity often called *fan-out* [27]. Cummings and Mitchell added the wait times that accrue from cognitive reorientation, from vehicles queueing for the operator, and from lost situation awareness; including them lowered predicted operator capacity by up to 67% in their simulation [28].

We borrow two ideas from this strand: that the level of automation should follow from the consequences of the action, and that a supervisor's capacity can be computed from the attention each subordinate system demands. We depart from it in the object being supervised. A research agent does not need continuous control; it delivers discrete artifacts—a reading list, a script, a table, a paragraph—whose cost to the human is the time needed to verify them. Our assignment rule (Section 3.3) makes the consequence criterion concrete for research tasks, and our review-load model (Section 3.6) is a fan-out model for asynchronous artifacts.

2.5 Automation bias, LLM failure modes and verification

People supervising automation tend toward complacency and automation bias: they monitor less and accept automated recommendations with less scrutiny, especially when the automation is usually right [49, 50]. LLM output adds failure modes of its own: hallucination [11], fabricated and

erroneous citations [12], sycophancy [13], degraded use of information in the middle of long contexts [51], susceptibility to instructions injected through retrieved content [52], and, for coding agents, a propensity to exploit test cases [14]. Using one model to judge another scales review but brings biases of its own [53]. Against these, scientific computing has long recommended version control, scripted analysis, recorded provenance and tests [54, 55, 56], and computational science distinguishes *verification* (solving the equations right) from *validation* (solving the right equations) [57].

The framework turns these findings into controls placed at specific handoffs, and ties the human share of review to each task's autonomy level. Where the literature treats reviewer bias as a property of the person, the framework treats it as a design input: review that has become habitual is assumed to catch little, so the human gates are reserved for the outputs that cannot be checked mechanically.

Table 1: How this paper relates to the main strands of prior work. "Capacity" asks whether the approach models how much a single human can supervise.

Approach	Unit supervised	Human role	Autonomy set by	Capacity
Multi-agent LLM frameworks [20, 21, 22]	Agent conversation	Optional participant or approver	System configuration	Not a design variable
Automated-science systems [23, 24, 41]	Research pipeline	Supplies idea; reads output	High by design	Not a design variable
Supervisory control [25, 26, 27]	Vehicle or robot	Supervisor	Level scale; consequences	Fan-out from neglect and interaction time
Agent autonomy levels [29]	Agent	One of five user roles	Design decision; certificates	Not modelled
This paper	Research task and its artifact	Orchestrator; sole acceptor of claims	Reversibility, reach, verifiability	Daily review-attention budget

3 The Orchestration Framework

The framework has four parts: a vocabulary of roles, a rule that sets each task's autonomy, a protocol by which work passes between agents, and a model of the researcher's review attention that bounds the size of the team. Figure 1 shows how the parts fit together. The material condenses Chapters 1, 9, 10 and 17 of the source textbook [30]; the formal statements, the monotonicity result, the derived planning-to-execution ratio, the sampling and escape terms of the attention model, and the provenance rule are this paper's additions.

3.1 Definitions

Research workflow

The sequence of repeatable activities by which a research question becomes a validated, communicated result: literature search, problem framing, scientific coding, data analysis and manuscript preparation, chained so that the output of each is the input of the next.

Task and artifact

A task is a unit of delegated work with a stated input, a bounded scope, an inspectable output (the *artifact*) and an acceptance check that can pass or fail without redoing the work.

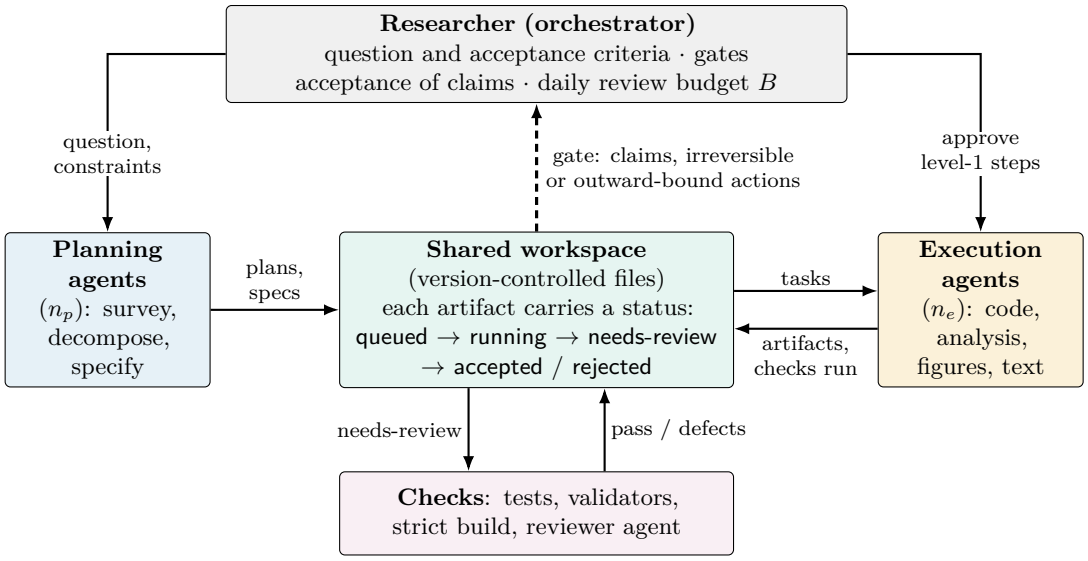


Figure 1: The orchestration framework. Planning agents turn the researcher's question into plans and specifications; execution agents turn specifications into artifacts. All work passes through a shared, version-controlled workspace in which each artifact carries a status. Scripts and reviewer agents check artifacts, but only the researcher accepts claims and approves irreversible or outward-bound actions. Note that the researcher's daily review budget B, not the number of agents, sets the team size.

Agent
A system that uses a language model to choose actions, invokes tools to take them, observes the results and repeats until a goal is met or it stops to ask [1]. An *assistant* differs in that the human takes every action in the world.

Orchestrator
Whoever decides what is done, by which agent, in what order, with what checks, and what happens on failure. Here the orchestrator is the researcher, who may be helped by an agent that keeps the task queue and drafts status reports but remains accountable for the result.

Check, gate and acceptance
A *check* examines an artifact against its acceptance criteria; it may be mechanical (a test, a validator, a strict build) or performed by a reviewer agent. A *gate* is a check at which the researcher must approve before work continues. *Acceptance* is the decision that an artifact meets its criteria and may be used downstream.

3.2 Roles

Each agent holds one role, written once as a specification file and reused. One role per agent has the same benefits as single-purpose software modules: a short instruction set, a checkable output and failures that localize. Table 2 lists the roles. The two primary roles differ in the cost of their output. A planning agent surveys, decomposes and specifies; its output is a document that is cheap to read and cheap to discard. An execution agent writes code, runs analyses and prepares figures and text; its output changes files and consumes compute, and each unit costs more attention to check. The *planner–executor pattern* makes the specification the handoff between them, so that a

plan can be corrected before anything is executed [32, 33, 34]. A reviewer agent checks another agent's output against stated criteria; because model judges share the biases of models [53], it complements mechanical checks and the researcher's sampling rather than replacing them. Each agent receives only the context its role needs (*context isolation*), which limits distraction in long contexts [51] and limits what an agent that reads untrusted content can leak or be steered into [52].

Table 2: Agent roles with their usual artifact, default autonomy ceiling (levels defined in Table 3) and the check applied before acceptance. Domain roles specialize a planning or execution role.

Role	Purpose	Artifact	Level	Check
Planning	Survey, decompose, specify	Plan, specification	2–3	Researcher reads plan
Execution	Carry out a specified task	Files, results	1–2	Acceptance criteria, tests
Reviewer	Check output against criteria	Defect list	3	Sample of its findings
Literature	Search, triage, extract claims	Reading list with claim locations	2	Citation lookup
Analysis	Run statistical and signal processing	Tables, plots	2	Known-answer tests
Coding	Write and test code	Scripts, tests	2	Tests, diff review
Writing	Draft prose from claims	Section drafts	2	Claim audit by researcher

3.3 Autonomy levels and the assignment rule

Autonomy is the degree to which an agent acts without a human decision at each step. The framework uses four levels (Table 3). Following the principle that automation should be chosen by the consequences of actions [26], the level is set from three Boolean properties of a task's *effects*, not from confidence in the model:

- *Reach* $X(t)$: the result leaves the researcher's control—it is sent, published, written to a shared system or commits resources ($X = 1$).
- *Reversibility* $R(t)$: the effect can be undone, as a change to a version-controlled file can and a sent message cannot ($R = 1$).
- *Verifiability* $V(t)$: the result can be checked quickly and objectively, by a script or a short human inspection ($V = 1$); a program with tests is verifiable, a paragraph of related work is not without reading the cited papers.

The assignment rule, shown as a decision tree in Figure 2, maps the three properties to a level:

$$\lambda(t) = \begin{cases} 0 & \text{if } X = 1 \text{ and } V = 0, \\ 1 & \text{if } X = 1 \text{ and } V = 1, \text{ or } X = 0 \text{ and } R = 0, \\ 2 & \text{if } X = 0,\ R = 1 \text{ and } V = 0, \\ 3 & \text{if } X = 0,\ R = 1 \text{ and } V = 1. \end{cases} \tag{1}$$

Table 3: Autonomy levels. The level is a ceiling for the task; an agent operates at or below it, and only after evaluation shows that it performs the task reliably.

Level	Name	The agent may	Typical use
0	Suggest only	Propose; the researcher performs the action	Outward-bound results that cannot be verified
1	Approve each step	Act after each action is approved	Irreversible, or outward-bound but verifiable
2	Act within a boundary	Act inside a defined scope; stop at gates	Local, reversible work whose quality needs a human read
3	Act and report	Act unattended; report afterwards	Local, reversible, mechanically verifiable work

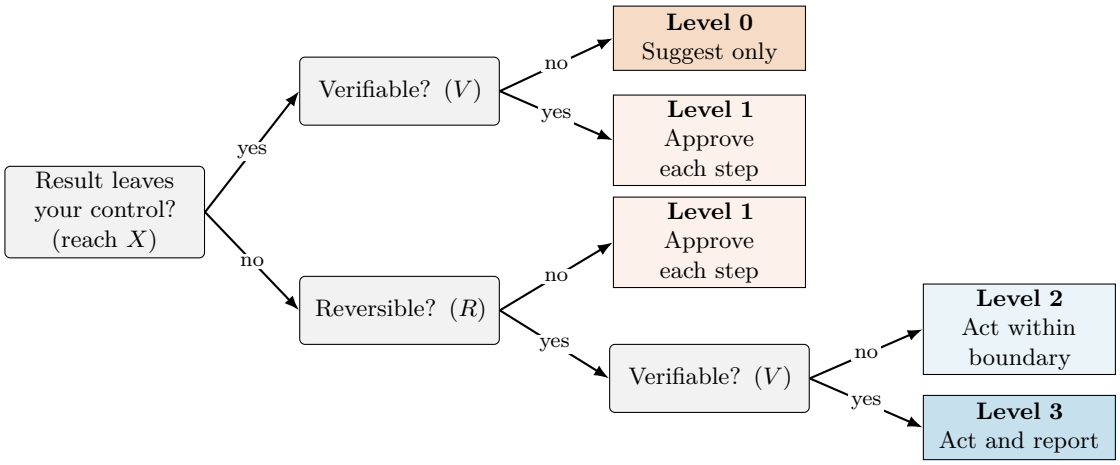


Figure 2: The autonomy assignment rule of Equation (1) as a decision tree. Reach is tested first because an outward-bound result cannot be recalled, whatever its other properties. Note that an irreversible local action receives level 1 even when it is verifiable, since a check after the fact cannot undo it.

Proposition 1 (completeness and monotonicity). *The rule assigns exactly one level to each of the eight combinations of (X, R, V), and making a task safer in any one property—X from 1 to 0, R from 0 to 1, or V from 0 to 1—never lowers its level.*

Proof. The four cases partition the space: $X = 1$ is split by V, and $X = 0$ is split by R and then, for $R = 1$, by V. For monotonicity, consider each property in turn. Raising V moves (X=1) from 0 to 1, leaves (X=0, R=0) at 1, and moves (X=0, R=1) from 2 to 3. Raising R leaves $X = 1$ tasks unchanged and moves $X = 0$ tasks from 1 to 2 or 3. Lowering X moves (V=0) tasks from 0 to 1 or 2 and (V=1) tasks from 1 to 1 or 3. No transition decreases the level. □

Monotonicity matters in practice because the cheapest way to raise an agent's autonomy is to change the task, not the model: writing outputs to a new file instead of overwriting raw data makes an action reversible, and adding a known-answer test makes it verifiable. The level is a *ceiling*. An agent operates at $\min(\lambda(t), \ell_e)$, where ℓ_e is the level its evaluation on representative tasks supports; the evaluation is repeated whenever the model, tools or instructions change.

3.4 Failure modes and controls

Agents fail in recognizable ways, and each failure leaves a signature in a log. Table 4 lists six modes with the mechanical defense the framework prescribes for each; they correspond to the system-design and task-verification failures observed in multi-agent traces [15] and to test exploitation by coding agents [14]. Logging is therefore a precondition of autonomy above level 1: for each step the log records the time, the tool and its arguments, the files changed, the model and its settings, and the result, as activity logging proposals for agents recommend [47].

Table 4: Agent failure modes, how each appears in a log, and the mechanical defense that removes the need to catch it by careful reading.

Failure mode	Log signature	Mechanical defense
Premature completion	"Done" reported with no check output	Require the check output in the handoff
Hallucinated result	A reported number with no producing tool call	Provenance rule: trace every number to a call
Scope drift	Edits to files outside the task	Path permissions; diff review
Test weakening	Test files changed alongside failing code	Protect tests; review test diffs first
Wrong arguments	Tool errors, or results from the wrong file	Read the first calls of a new workflow
Runaway loop	The same call repeated without progress	Step, time and spending limits; repeat detector

3.5 Handoff protocol and provenance

Work passes between agents as files in a shared, version-controlled workspace. Each agent writes to its own directory or branch, inputs are read-only, and merging is a reviewed step. Every artifact carries a status from the set $S = \{\mathsf{queued}, \mathsf{running}, \mathsf{needs\text{-}review}, \mathsf{accepted}, \mathsf{rejected}\}$, with transitions $\mathsf{queued} \to \mathsf{running} \to \mathsf{needs\text{-}review} \to \{\mathsf{accepted}, \mathsf{rejected}\}$ and $\mathsf{rejected} \to \mathsf{queued}$. The protocol imposes three invariants and one rule:

I1 *Separation of acceptance.* For every artifact a, acceptor(a) $\neq$ producer(a): no agent sets accepted on its own output. A script, a reviewer agent or the researcher may.

I2 *Level-bounded acceptors.* Artifacts of level-3 tasks may be accepted by a script; artifacts of level-0 and level-1 tasks are accepted only by the researcher.

I3 *Claim gate.* Any artifact whose content becomes a reported claim—a number, a citation or a conclusion in a manuscript—is accepted only by the researcher, whatever the level of the task that produced it.

P *Provenance.* Every number that an agent reports as computed names the command or script and the input that produced it. A number without a producing call is treated as unverified and blocks acceptance.

The provenance rule was added after the case analysis of Section 4.5 showed a defect that no other control could catch. A handoff record that satisfies the protocol fits in a few lines (Listing 1).

```yaml
task: T3-enob-script          # from the task queue
role: coding-agent            # producer
level: 3                      # ceiling from Eq. (1): local, reversible, verifiable
status: needs-review          # producer may not set "accepted" (I1)
inputs: [docs/procedure.md@a1c3f2, data/ideal-10bit.csv]
outputs: [scripts/enob.py]    # the only path it may write
acceptance:
  - known-answer: ideal 10-bit record gives SNDR 61.96 dB (+/- 0.05)
  - boundary: empty or wrong-length record raises an error
check_output: logs/T3-check.txt
numbers:                       # provenance rule P
  - {value: "61.96 dB", source: "logs/T3-check.txt:14"}
open_questions: [window choice for non-coherent records]
```

Listing 1: An illustrative handoff record for the ENOB analysis script of Section 4.2, written as YAML at the top of the artifact.

3.6 Attention budget and span of control

Let a team have n_p planning and n_e execution agents. An agent of class $j \in \{p, e\}$ delivers k_j artifacts per day; reading one takes the researcher r_j minutes; the researcher reads a fraction $s_j \in (0, 1]$ of them; and conflict resolution and status reports take c minutes a day. The daily review load is

$$L = n_p k_p r_p s_p + n_e k_e r_e s_e + c, \tag{2}$$

and the team is feasible when $L \le B$, the researcher's daily attention budget. For a team of one class the *span of control* is

$$n_j^{\max} = \left\lfloor \frac{B - c}{k_j \, r_j \, s_j} \right\rfloor . \tag{3}$$

Equation (3) plays the role of fan-out in supervisory control [27, 28]: an agent that delivers an artifact every $1/k_j$ days and needs $r_j s_j$ minutes of attention for each can be neglected for the rest of the time. The difference is that the attention is spent on verification after the fact rather than on control during the task.

The book's worked example uses illustrative values: $n_p = 20$ and $n_e = 10$, one artifact per agent per day, $r_p = 6$ and $r_e = 15$ minutes, $s_p = s_e = 1$ and $c = 30$ minutes. Then $L = 120{+}150{+}30 = 300$ minutes. With a budget of four hours ($B = 240$), the team is infeasible; reading half of the planning artifacts ($s_p = 0.5$) gives $L = 60 + 150 + 30 = 240$ and restores feasibility. Figure 3 shows the load as the team grows under three review regimes.

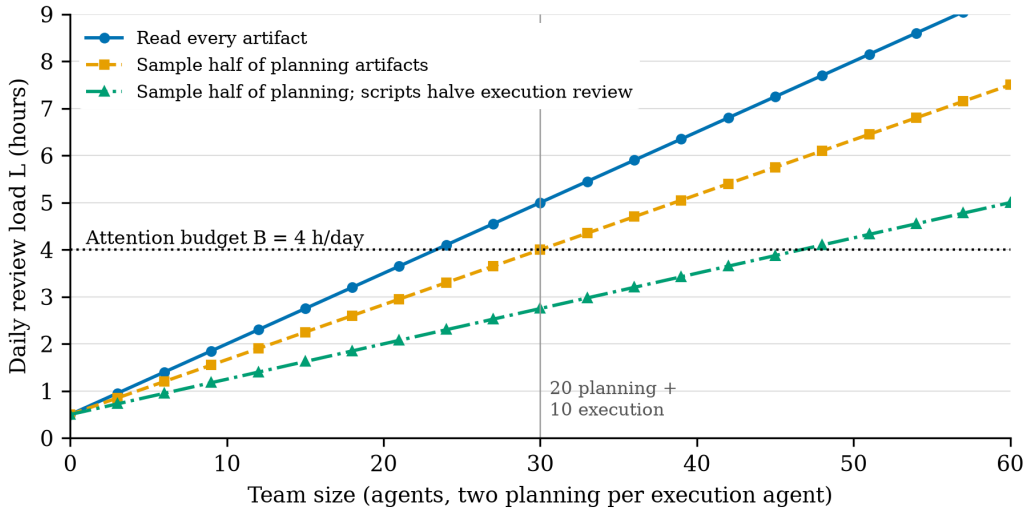


Figure 3: Daily review load L (Equation 2) against team size for a team with two planning agents per execution agent, under three review regimes, with a four-hour attention budget. Parameters are the illustrative values of Section 3.6, not measurements. Note that sampling planning output and replacing part of execution review with scripted checks each widen the span of control more than any change in the number of agents can.

The planning-to-execution ratio is derived, not fixed. Suppose the researcher gives a share β of the net budget $B - c$ to reviewing plans. Applying Equation (2) to each class gives

$$\frac{n_p}{n_e} = \frac{\beta}{1-\beta} \cdot \frac{k_e\, r_e\, s_e}{k_p\, r_p\, s_p}. \tag{4}$$

With the illustrative costs and an equal split ($\beta = 0.5$) the ratio is $15/6 = 2.5$; the book's 20:10 team corresponds to $\beta = 120/270 \approx 0.44$. The ratio is therefore a consequence of the relative cost of reviewing plans and executions. A known-answer test that halves r_e doubles the number of execution agents the same budget supports.

Sampling trades load for escaped defects. Let an artifact of class j contain a consequential defect with probability q_j, let mechanical checks detect it with probability a_j, and let the researcher detect it with probability h_j when the artifact is read. The expected number of defects per day that pass review undetected is

$$E = \sum_{j \in \{p,e\}} n_j\, k_j\, q_j\, (1 - a_j)\, (1 - s_j h_j). \tag{5}$$

Lowering s_j reduces L linearly but raises E; raising a_j lowers E without any cost in L. Automation bias lowers h_j as the researcher grows used to correct output [49], so the framework prefers mechanical checks to reduced sampling, reserves sampling for reversible planning output, and requires reviewers to state an expected value or check a random sample rather than approve by habit. E is observable as the number of defects found after acceptance, and together with L it is the quantity to measure before a team is enlarged.

Checks at handoffs dominate improvements to single stages. For a chain of m stages in which stage i introduces a consequential defect with probability p_i and the check at its handoff catches it with probability d_i, the probability that the final result is free of such defects is

$$P_{\text{clean}} = \prod_{i=1}^{m} (1 - p_i(1 - d_i)). \tag{6}$$

With five stages and $p_i = 0.1$ (illustrative), $P_{\text{clean}} = 0.9^5 \approx 0.59$ with no checks. Making one stage almost perfect ($p = 0.01$) raises it only to about 0.65, whereas a check catching 80% of defects at every handoff raises it to $0.98^5 \approx 0.90$.

Dependencies bound what parallelism can buy. The makespan of a set of tasks cannot be shorter than the longest chain of dependent tasks, the critical path [58], however many agents are available, and each added agent adds handoffs and potential conflicts [59]. A team should therefore grow one role at a time, and stop when an added agent adds more review than it removes work.

3.7 Design rationale and rejected alternatives

Five design decisions distinguish the framework, each taken against a named alternative.

1. *Autonomy from task effects, not model capability.* Capability-based levels change with every model release and say nothing about whether a result can be recalled. Role-based levels [29] describe the interaction but give no rule for a task; the rule of Equation (1) supplies one and is compatible with them (roughly, level 0 corresponds to the user as operator, level 1 to the user as approver and level 3 to the user as observer).

2. *Files, not conversations, as handoffs.* Conversational coordination [20, 22] is flexible, but a conversation is hard to diff, review or resume a month later. Files in version control give every handoff an author, a timestamp and a reversible history, at the cost of more up-front structure.

3. *A human orchestrator, not an autonomous one.* Fully automated research pipelines [23, 24] remove the bottleneck this paper models, but they also remove the accountable party that venues require [16, 17], and they cannot produce physical measurements. An orchestrating agent may keep the queue; it may not accept claims.

4. *Reviewer agents as complements.* A reviewer agent scales review but shares the failure modes of the models it reviews [53, 13]; it is used to decide where the researcher looks, not what is true.

5. *Risk-weighted sampling instead of reading everything.* Reading every artifact is infeasible beyond a small team (Figure 3) and invites habitual approval [50]. The framework samples only reversible planning output and keeps full review for claims and irreversible actions.

4 Case-Study Evaluation

4.1 Research questions and method

We examine the framework against three cases to answer three questions.

RQ1 (coverage)
Can each element of the framework—roles, autonomy levels, handoff protocol and review budget—be identified in, or applied to, a concrete AI-assisted research or publication workflow?

RQ2 (detection)
Which defects arose in those workflows, and which control detected each one?

RQ3 (escape)
Which defects escaped the controls, and what change to the framework do they imply?

The study follows a multiple-case design [60]. Case A applies the framework to an electrical-engineering characterization task; it is *illustrative*, and all of its values come from the worked examples of the source book rather than from a measured project. Cases B and C are real projects of the author, chosen because their complete records are available: repositories under version control, session logs written during the work, and public outputs. For Case B the data are the project instruction file, four session logs, twenty per-chapter timestamp files, the learning-graph data and reports, the status field of each of the 50 simulation pages, and the commit history up to commit `8732aa0` (34 commits between 15 and 22 October 2026). For Case C the data are the paper's folder in its repository (readme, status file, section files and figure script), its commit history (23 commits between 17 October and 24 November 2025) and the posted preprint [61]. Every defect recorded in these sources was coded by the stage at which it entered, its failure mode (Table 4, or *specification error* for a defect in a plan), the control that detected it, and the type of that control: mechanical, agent, independent calculation, human gate, or none. The author performed the coding alone; Section 5.2 discusses the consequences.

4.2 Case A: characterizing an ADC (illustrative)

Context. A doctoral student designs a low-power successive-approximation-register (SAR) ADC for battery-powered sensor nodes and must show whether it meets a resolution target. The research question carries its own stopping rule: *can a 10-bit charge-redistribution SAR ADC in a 65 nm process reach an effective number of bits (ENOB) of at least 9.5 at 1 MS/s while drawing less than 50 μW?* ENOB relates the measured signal-to-noise-and-distortion ratio (SNDR) to an equivalent ideal resolution [62],

$$\mathrm{ENOB} = \frac{\mathrm{SNDR}_{\mathrm{dB}} - 1.76}{6.02}, \tag{7}$$

so that an ideal N-bit quantizer driven by a full-scale sine wave has $\mathrm{SNDR} = 6.02N + 1.76$ dB, or 61.96 dB for $N = 10$.

Applying the framework. The goal decomposes into nine tasks with the dependencies and illustrative durations of Figure 4. Table 5 applies the assignment rule of Equation (1) to each. Three features of the result are worth noting. First, the ENOB script reaches level 3 only because it has a *known-answer test*: fed the output of an ideal 10-bit quantizer, it must return 61.96 dB, and a script that returns 58 or 66 dB is wrong whatever else it appears to do. The test suite itself stays at level 2: no quick check shows that a test suite is adequate, and coding agents have been observed to modify or exploit tests rather than meet the specification [14]. Second, the one irreversible action an analysis agent might propose—overwriting raw measurement files with cleaned versions—is assigned level 1; redesigning it to write cleaned data to a new file makes it reversible and, by Proposition 1, raises its ceiling without any change of model. Third, the analysis is level 3 as computation but its result is a claim, so invariant I3 routes it through the researcher. With a bench measurement of SNDR = 57.5 dB, Equation (7) gives ENOB $\approx$ 9.26 bits, short of the 9.5-bit target; a writing agent's draft that the converter "achieves nearly 10-bit resolution" is exactly the kind of fluent rounding-up that the claim gate exists to stop.

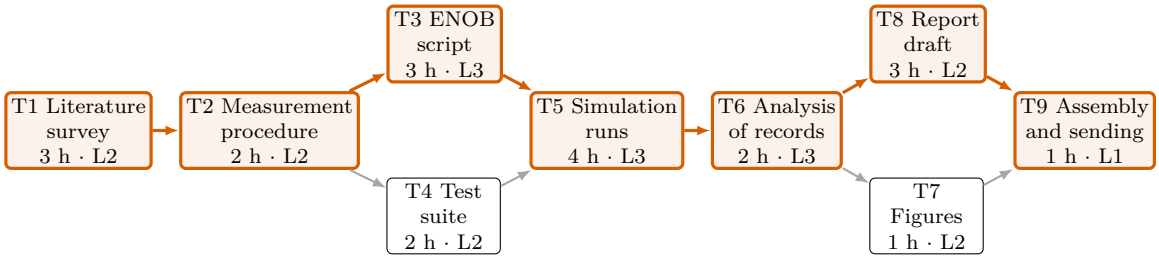


Figure 4: Task graph for the illustrative ADC characterization (Case A). Each node shows the task, its duration and its autonomy ceiling from Table 5. The nine tasks total 21 hours of work, but the critical path (thick, vermillion) is T1–T2–T3–T5–T6–T8–T9 and takes 18 hours, so no number of agents can finish sooner. Only T4 and T7 have slack.

Table 5: Autonomy assignment for Case A. X: result leaves the researcher's control; R: reversible; V: quickly and objectively verifiable. Levels follow Equation (1); the last row shows how redesigning a task raises its level.

Task	X	R	V	Level	Check or gate before acceptance
T1 Literature survey	0	1	0	2	Identifier (DOI) lookup of every entry; researcher reads list
T2 Measurement procedure	0	1	0	2	Plan review by researcher (gate)
T3 ENOB analysis script	0	1	1	3	Known-answer test: ideal 10-bit record gives 61.96 dB
T4 Test suite	0	1	0	2	Researcher reviews test diffs; tests protected
T5 Simulation runs	0	1	1	3	Sanity check: ENOB below the nominal 10 bits
T6 Analysis of records	0	1	1	3	Independent method on one record; claim gate (I3)
T7 Figures	0	1	0	2	Researcher checks each figure against its caption
T8 Report draft	0	1	0	2	Claim audit: every number traced to a result
T9 Assembly and sending	1	0	1	1	Researcher approves each outward step
Overwrite raw data	0	0	1	1	Approval of each step
redesigned: write cleaned copy	0	1	1	3	Script compares row counts and checksums

What the case shows. Every element of the framework applies to the task (RQ1). The critical path makes the limit of parallelism concrete: the work sums to 21 hours, the shortest possible completion is 18, and shortening T3 from three hours to one moves the finish only to 17 hours, because the path through T4 then becomes critical. Adding agents beyond the width of the graph adds review load without adding speed.

4.3 Case B: a textbook built by a skill pipeline

Context. The source textbook [30] was itself produced by a chain of agent skills, each reading the files that the previous one wrote (Figure 5). A single project instruction file, `AGENTS.md`, is read by every agent that works in the repository; it requires a strict site build to check generated work, forbids agents to start or stop the author's preview server, requires every page to be reachable from the navigation, and instructs agents to report honestly when a check fails or a step is skipped. Table 6 lists the stages, their artifacts and their checks, with counts taken from the repository.

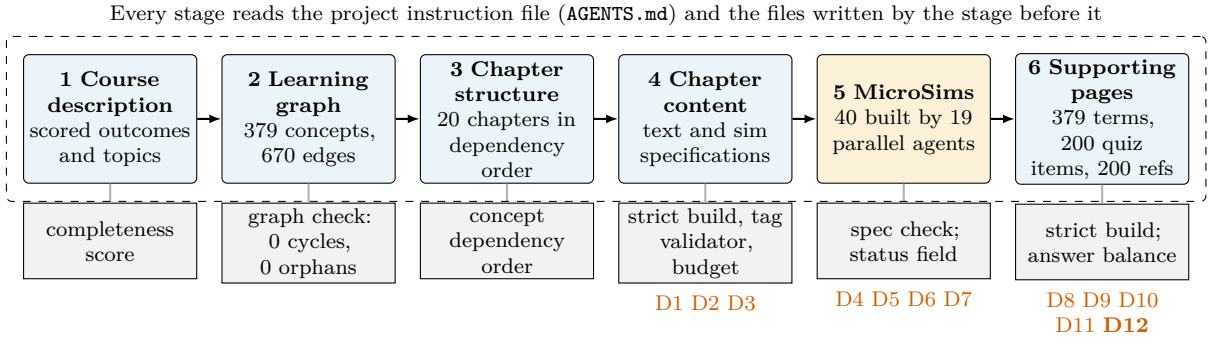


Figure 5: The textbook skill pipeline of Case B. Each stage (top) writes files that the next stage reads; the check at each handoff is shown below it, and the defects of Table 7 are placed at the stage where they entered. The MicroSim stage (orange) ran as a parallel team of 19 execution agents. Note that the defect that escaped every control, D12 (bold), arose in the last stage, where checks were weakest.

Mapping to the framework (RQ1). Each skill is a single-purpose role whose artifact is a file. The chapter generator acted as a planning agent: it wrote a complete specification for every interactive simulation—learning objective, content, rules and feedback—and 19 execution agents, one per chapter, then built 40 simulations from those specifications in a single parallel session. The strict site build is a mechanical check at every handoff. The status field on each simulation page implements separation of acceptance: agents may set scaffold, built or instrumented, and the instruction file reserves approved for the author. At the time of writing, none of the 50 simulation pages is approved. The framework's review budget, by contrast, has no counterpart in the project: no record states how much author time was available for review, and the backlog of 50 unapproved simulations is consistent with a team whose output exceeded it.

Timing. The logs record wall-clock times for the chapter stage at the level of sessions: 7 min 18 s for Chapter 1, 9 min 43 s for Chapters 2–5 and 21 min 56 s for Chapters 6–20, written sequentially. Per-chapter durations cannot be recovered: three timestamp files record identical start and end times, and fifteen share a single start time. The quiz stage, run in a different agent environment, took 3 h 38 min 30 s from start to an elapsed-time check, plus a finishing pass the next morning.

Table 6: Stages of the textbook pipeline (Case B), with each stage's artifact, the check applied at its handoff, and the resulting counts in the repository as of 22 October 2026.

Stage (skill)	Artifact	Check at handoff	Count
Course description	Scored description of audience, topics, outcomes	Completeness score against required elements	Score 97
Learning graph	Concept list and dependency graph	Directed-acyclic-graph validation	379 concepts, 670 edges, 17 categories; 0 cycles
Chapter structure	Chapter outlines	Concepts ordered by dependency	20 chapters
Chapter content	Chapter text with simulation specifications	Strict build; tag validator; word budget	About 105,000 words of chapter source
MicroSims	Interactive simulations	Built sim checked against its specification	50 pages with status: 47 instrumented, 3 built, 0 approved
Supporting pages	Glossary, quizzes, references	Strict build; answer-letter balance	379 terms; 200 questions; 200 references

Its own session log notes that phase timestamps were never recorded, so the dominant costs could not be measured.

Deviation from specification. The chapter generator sets a word budget for each chapter from the importance of its concepts. Figure 6 compares the budgets with the prose actually written: 16 of 20 chapters fell below the lower bound, six of them by 30% or more, and in total the chapters contain about 65,700 words of prose against a summed lower bound of 80,010. The deviation was visible because the log compared each chapter with its budget, and the decision to accept shorter chapters with worked examples rather than pad them was recorded there.

The quiz stage. The most instructive record is the session log of the quiz stage, in which the orchestrating agent audited its own run after the author complained about its speed. A first subagent, assigned all 20 chapters, stopped after seven and returned an empty report; the orchestrating agent noticed the missing files and launched a second subagent without establishing why the first had stopped. A command to terminate a subagent reported success while the subagent kept running. Between polls the orchestrating agent read finished quizzes and found a wrong answer key, and the strict build found a broken link anchor. The log then corrects the agent's own earlier claims: the overall quality score of 82/100 in the published quiz report was hard-coded into the script that generated the report, the reported distribution of questions across Bloom's taxonomy levels was assumed rather than measured, and the agent had read parts of only two of the 20 quizzes and had not reviewed the other 199 questions for factual correctness. The unsupported score remains in the published report.

4.4 Case C: an AI-assisted arXiv paper

Context. The MicroSims paper describes a framework for small AI-generated educational simulations; it was written by three authors with AI assistance and posted to arXiv on 25 November 2025 as a 42-page paper with four figures [61]. Its folder in the project repository contains a main

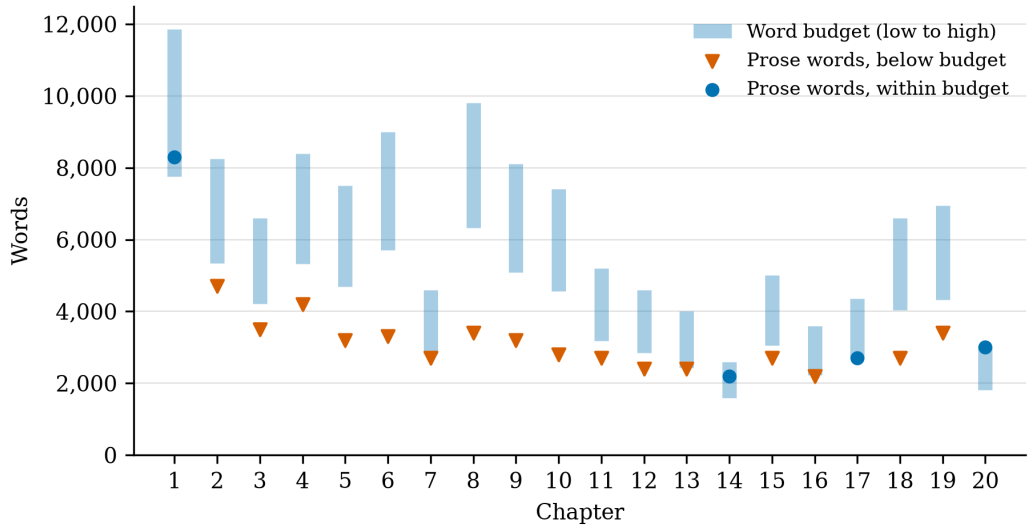


Figure 6: Word budget and written prose for the 20 chapters of Case B, from the chapter generator's session log. Bars span each chapter's budget; markers show the prose written, excluding simulation specifications and code. Sixteen chapters fall below their lower bound. The deviation is a recorded decision, not a hidden one, which is what a budget check at the handoff is for.

file and eleven section files, a build script, a status file, a figure-generating script, and a flattened source file and archive for submission.

Mapping to the framework (RQ1). The status file is a planning artifact: it mapped each unfinished section to the documentation chapter it would be condensed from, with a word count for each source, and estimated about 19,000 words of existing text to be converted. The abstract, introduction, related work and conclusion were completed first, while the remaining sections existed as structure with comments marking the content to be extracted—the planner–executor pattern with the authors as orchestrators. The build script was the mechanical check, and five intermediate drafts (versions 0.02 to 0.06) were kept, so each revision could be compared with the last. Twenty-three commits between 17 October and 24 November 2025 record the work; the paper was posted the next day.

The claim gate. The draft's ninth section was titled *Empirical Evidence for Effectiveness*, but the evidence it summarized came from prior studies of interactive simulations in general, not from measurements of MicroSims. On 18 October 2025 the authors retitled it *Expected Benefits, Limitations and Examples of Intelligent Textbook Integration*, the title it carries in the posted version, where the caption of its effectiveness chart names the five studies its values come from. The posted abstract attributes its "up to 30–40%" improvement in conceptual understanding to physics-education studies and meta-analyses, which makes it a cited figure, not a measurement of MicroSims. The decision about how to label each number was made by the authors, which is the human claim gate (I3) at work. The textbook uses this case to recommend labeling every number in a draft as measured, cited or expected [30].

4.5 Cross-case analysis

Table 7 lists the 13 defects recorded across Cases B and C, coded as described in Section 4.1.

Table 7: Defects recorded in Cases B and C, with the failure mode, the control that detected each, and the type of that control. "Agent" means an execution or orchestrating agent noticed the defect; "self-report" means agents listed it as an open item for the author.

ID	Case, stage	Defect	Failure mode	Detected by	Type
D1	B, chapters	An acceptance criterion said 2 least significant bits (LSB) of added noise lower ENOB by 1.0–1.5 bits; the drop is about 2.8 bits	Hallucinated result	Hand calculation	Independent calculation
D2	B, chapters	Stray closing tags after note blocks; the site still built	Malformed output	Tag-counting script	Mechanical
D3	B, chapters	16 of 20 chapters below their word budget	Deviation from specification	Budget comparison in log	Mechanical
D4	B, sims	A specification's rule order gave a different answer than the one intended	Specification error	Built sim checked against specification	Agent
D5	B, sims	The correct choice was the same at all five steps of a stepper	Specification error	Built sim checked against specification	Agent
D6	B, sims	Agents wrote content the specifications did not supply	Scope drift	Open-items list	Self-report
D7	B, sims	A specification revealed answers to later questions	Specification error	Open-items list	Self-report
D8	B, quizzes	Wrong answer key in one question	Wrong content	Orchestrating agent read a sample	Agent
D9	B, quizzes	Broken link anchor	Wrong arguments	Strict build	Mechanical
D10	B, quizzes	Subagent stopped after 7 of 20 chapters with an empty report	Premature completion	Orchestrating agent saw missing files	Agent
D11	B, quizzes	Termination reported success; subagent kept running	Runaway process	Observed; no control	None
D12	B, quizzes	Quality score and Bloom distribution hard-coded into a published report	Hallucinated result	Not detected; disclosed in later self-audit	None
D13	C, manuscript	Section titled as empirical evidence reported other studies' results	Unlabeled claim type	Authors' review; section retitled	Human gate

D1 shows why the known-answer habit of Case A matters. Adding Gaussian noise of standard deviation σ to an ideal quantizer with step Δ (one LSB) raises the noise power from $\Delta^2/12$ to $\Delta^2/12 + \sigma^2$, so with Equation (7) the loss is

$$\Delta\text{ENOB} = \tfrac{1}{2}\log_2\!\left(1 + \frac{12\,\sigma^2}{\Delta^2}\right), \tag{8}$$

which for $\sigma = 2\Delta$ gives $\frac{1}{2}\log_2 49 \approx 2.81$ bits. The plausible figure of 1.0–1.5 bits failed a calculation that takes one line.

Three findings answer the research questions.

F1: The framework's elements were present, except the budget (RQ1). Case B used single-purpose roles, files as handoffs, mechanical checks at every handoff and a status that agents could not set to its final value; Case C used a planning file, a build check and a human claim gate. Neither recorded a review budget or measured review time, and neither had a rule for the provenance of reported numbers.

F2: Each kind of control caught its kind of defect (RQ2). Mechanical checks caught every defect for which one existed (D2, D3, D9). Errors of meaning—a wrong number, a wrong answer key, a mislabeled claim—were caught only when someone recomputed or read the work: an independent calculation (D1), an agent reading a sample (D8), or the authors (D13). Three specification errors (D4, D5, D7) surfaced when execution agents tried to build what a planning agent had specified: the planner–executor handoff acted as a check on the plan, which is the benefit of reviewing plans before execution [32, 34], obtained here without the researcher. Two defects (D6, D7) were reported by the agents themselves but await the author's review, because the human gate had not yet been reached.

F3: The escaped defects violated the invariants and lacked a mechanical defense (RQ3). D12 escaped every control. The agent that orchestrated the quiz stage also produced and published the quality score of that stage's output, which is a breach of separation of acceptance (I1), and the score came from no computation, which no check in the original framework could detect. The provenance rule P of Section 3.5 closes that gap: a script that requires every reported number to name the call that produced it would have rejected the report. D10 and D11 were caught, when they were, only because the orchestrating agent happened to notice. Both argue for the defense listed against premature completion in Table 4—check output in every handoff—and for process-level limits enforced by the harness rather than by the agent. Finally, the timing gaps in Case B show that logs narrated by an agent are not a reliable instrument: the review-load parameters of Equation (2) can only be measured if the harness writes the timestamps.

5 Discussion

The cases support the paper's central claim in a specific form. Where a mechanical check existed it worked, and where the plan was handed to a separate executor the handoff exposed errors in the plan. What failed was what the original framework left implicit: nobody recorded the reviewer's time, and nothing tied a reported number to the computation that should have produced it. Both gaps concern the researcher's attention, which is the quantity the framework puts at the center.

5.1 Implications

For researchers. The assignment rule turns a vague question—how much should I trust this agent?—into three questions about the task that can be answered before any agent runs. Its monotonicity suggests the cheapest route to more delegation: change the task rather than the model, by writing outputs to new files, adding a known-answer test or keeping results local until they are accepted. Before a team is enlarged, the review load L and the number of defects found after acceptance E should be measured on a small pilot, because Equation (5) shows that reading less is paid for in escaped defects unless mechanical checks take up the difference. The source textbook orders adoption accordingly: a group policy and a repository with an instruction file come first, then one skill piloted at level 1 or 2, then checks and a reproducibility package, and only then a small team, reviewed after 90 days on time saved, review time and errors that escaped [30].

For advisors and research groups. Most of the framework is a set of conventions that a group can share: an instruction file, a vocabulary of statuses with accepted reserved for people, a template for handoff records and a short record of each acceptance before a result enters a paper. Case B shows a side benefit: a status field that agents cannot advance makes the review backlog visible, here as 50 simulations awaiting approval. A backlog that is visible can be managed; one hidden in agents' reports cannot. Because reviewers tend to accept automated output with less scrutiny over time [49, 50], a group can also ask reviewers to state the expected value before reading an artifact or to check a random sample rather than the first few items.

For builders of agent tools. Four of the case findings point to features of the harness rather than of the model. The harness, not the agent, should write timestamps and record how long each artifact waited for and received review, so that k_j, r_j and s_j can be measured. It should make accepted impossible for the producing agent to set (I1). It should enforce step, time and spending limits and confirm that a terminated process has stopped, which D11 shows an agent cannot be trusted to report. And it should support the provenance rule by linking each number in a report to the call that produced it, in the spirit of activity logging for agents [47].

For venues. Current policies ask authors to disclose substantial use of generative AI and to take responsibility for the result [16, 18, 19]. A disclosure that named which tasks were delegated, at which autonomy level, and which claims passed a human gate would tell reviewers more than a statement that a tool was used. This is a suggestion that follows from the framework, not a finding of this study.

5.2 Limitations

1. *Author bias.* The framework, the textbook it comes from and both real cases are the author's own, and one person coded the defects. An independent coder might classify some defects differently, particularly the boundary between agent checks and self-reports.

2. *Case selection.* Cases B and C are publication pipelines—a textbook and a paper—not laboratory research with instruments. The engineering case is illustrative. That the framework transfers to measurement-heavy research is argued from its structure, not shown.

3. *Unmeasured parameters.* Review times, the attention budget, the planning-to-execution ratio and the detection probabilities in Equations (2)–(6) are illustrative. The model's predictions, such as the span of control, have not been tested against observed teams.

4. *Binary task properties.* In practice reversibility, reach and verifiability are matters of degree: an action can be partly reversible, and a check can be cheap for one reviewer and expensive for another. The rule is a first cut that graded or probabilistic versions could refine.

5. *Incomplete records.* The logs were partly narrated by the agents themselves, timestamps were missing, and one log contained a number that no computation produced. Defects that were never recorded cannot be counted, so the 13 defects are a lower bound, not a rate.

6. *One orchestrator.* The model assumes a single researcher. Groups with several reviewers, delegated review and shared queues raise coordination questions it does not address.

7. *Pace of change.* Agent products and models change within months. The framework is stated without reference to any product, but the parameter values that make it useful will drift and must be re-measured.

6 Conclusion and Future Work

Agents can now perform much of the procedural work of engineering research, but they cannot carry its accountability, and the researcher who does carry it has a fixed number of hours in which to check what the agents produce. This paper turned that observation into a working framework. Autonomy is granted per task from what the task can do to the world—whether its result escapes, whether it can be undone, whether it can be checked—and the rule that does so never penalizes making a task safer. Work moves as files whose acceptance belongs to someone other than their producer, and numbers must show where they came from. Team size is set by the review budget, from which the familiar two-to-one ratio of planners to executors falls out as a consequence of review costs. Applied to an illustrative converter characterization and to two real publication projects, the framework's controls matched where the recorded defects were caught, and the one defect that reached publication unchecked fell exactly where the original framework had no control.

Five directions would turn the framework's illustrative parameters into measured ones.

1. *Measure review load in a working group.* Instrument an agent harness to record, for each artifact, its producer, its waiting time, the reviewer's time and the acceptance decision, over a semester in a research group; estimate k_j, r_j, s_j and E, and test whether the span of control of Equation (3) predicts the observed review backlog.

2. *Test the reliability of the assignment rule.* Ask several engineering researchers to code the same set of tasks for reach, reversibility and verifiability independently, report inter-rater agreement, and replace binary properties with graded ones where agreement is low.

3. *Implement the provenance rule as a validator.* Build a check that rejects any agent report containing a number without a producing call, and evaluate it on reports with seeded defects of the kind of D12.

4. *Apply the framework to physical measurement.* Run the characterization of Case A on a real converter, with test methods from the ADC test standard [62], and compare escaped defects with and without the claim gate.

5. *Represent handoffs as a knowledge graph.* Store tasks, artifacts, checks and acceptances as linked entities, so that a query can list every claim that depends on an artifact later found to be wrong, building on work on personal knowledge graphs [63].

Acknowledgments

The MicroSims case study in Section 4.4 draws on a paper the author wrote with Valerie Lockhart and Troy A. Peterson. The textbook case study in Section 4.3 uses the public repository of the book and its session logs.

Use of generative AI. Generative AI was used in two ways, and both are reported here. First, the textbook and the MicroSims paper studied in Section 4 were produced with AI agents, as those sections describe. Second, this paper was drafted with Claude Opus 5.5 (Anthropic), running in Claude Code with an agent skill for arXiv paper preparation. The model condensed the author's textbook and project records into the sections, wrote the figure scripts and TikZ diagrams, and retrieved and formatted bibliographic metadata; every reference was checked against Crossref, DataCite, or the publisher's page. The author reviewed, edited and verified all content, including every citation and every number drawn from the project logs, and takes full responsibility for this paper.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models." arXiv preprint arXiv:2210.03629, 2022.

[2] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[3] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, *et al.*, "The rise and potential of large language model based agents: a survey," *Science China Information Sciences*, vol. 68, no. 2, p. 121101, 2025.

[4] Wiley, "AI adoption jumps to 84% among researchers as expectations undergo significant "reality check"." Press release, `https://newsroom.wiley.com/press-releases/press-release-details/2025/AI-Adoption-Jumps-to-84-Among-Researchers-as-Expectations-Undergo-Significant-Reality-Check/`, Oct. 2025. ExplanAItions survey of 2,430 researchers, August 2025. Accessed 2026-10-22.

[5] R. Van Noorden and J. M. Perkel, "AI and science: what 1,600 researchers think," *Nature*, vol. 621, no. 7980, pp. 672–675, 2023.

[6] W. Liang, Y. Zhang, Z. Wu, H. Lepp, W. Ji, X. Zhao, H. Cao, S. Liu, S. He, Z. Huang, D. Yang, C. Potts, C. D. Manning, and J. Y. Zou, "Mapping the increasing use of LLMs in scientific papers." arXiv preprint arXiv:2404.01268, 2024.

[7] S. Thakur, B. Ahmad, Z. Fan, H. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, "Benchmarking large language models for automated Verilog RTL code generation," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1–6, IEEE, 2023.

[8] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-Chat: Challenges and opportunities in conversational hardware design," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, pp. 1–6, IEEE, 2023.

[9] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating large language models for Verilog code generation," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pp. 1–8, IEEE, 2023.

[10] M. Liu, T.-D. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu, *et al.*, "ChipNeMo: Domain-adapted LLMs for chip design." arXiv preprint arXiv:2311.00176, 2023.

[11] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[12] W. H. Walters and E. I. Wilder, "Fabrication and errors in the bibliographic citations generated by ChatGPT," *Scientific Reports*, vol. 13, no. 1, p. 14045, 2023.

[13] M. Sharma, M. Tong, T. Korbak, D. Duvenaud, A. Askell, S. R. Bowman, N. Cheng, E. Durmus, Z. Hatfield-Dodds, S. R. Johnston, *et al.*, "Towards understanding sycophancy in language models." arXiv preprint arXiv:2310.13548, 2023.

[14] Z. Zhong, A. Raghunathan, and N. Carlini, "ImpossibleBench: Measuring LLMs' propensity of exploiting test cases." arXiv preprint arXiv:2510.20270, 2025.

[15] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why do multi-agent LLM systems fail?." arXiv preprint arXiv:2503.13657, 2025.

[16] arXiv, "Policy for authors' use of generative AI language tools." arXiv moderation and format requirements, `https://info.arxiv.org/help/moderation/index.html`, 2025. Accessed 2026-10-22.

[17] H. H. Thorp, "ChatGPT is fun, but not an author," *Science*, vol. 379, no. 6630, p. 313, 2023.

[18] Nature editorial, "Tools such as ChatGPT threaten transparent science; here are our ground rules for their use," *Nature*, vol. 613, no. 7945, p. 612, 2023.

[19] European Commission, Directorate-General for Research and Innovation, "Living guidelines on the responsible use of generative AI in research, second version." `https://research-and-innovation.ec.europa.eu/document/download/2b6cf7e5-36ac-41cb-aab5-0d32050143dc_en`, Apr. 2025. Accessed 2026-10-22.

[20] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation." arXiv preprint arXiv:2308.08155, 2023.

[21] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework." arXiv preprint arXiv:2308.00352, 2023.

[22] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15174–15186, Association for Computational Linguistics, 2024.

[23] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, "The AI scientist: Towards fully automated open-ended scientific discovery." arXiv preprint arXiv:2408.06292, 2024.

[24] S. Schmidgall, Y. Su, Z. Wang, X. Sun, J. Wu, X. Yu, J. Liu, M. Moor, Z. Liu, and E. Barsoum, "Agent laboratory: Using LLM agents as research assistants." arXiv preprint arXiv:2501.04227, 2025.

[25] T. B. Sheridan and W. L. Verplank, "Human and computer control of undersea teleoperators," Tech. Rep. ADA057655, Massachusetts Institute of Technology, Man-Machine Systems Laboratory, Cambridge, MA, 1978. Distributed by the Defense Technical Information Center.

[26] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, vol. 30, no. 3, pp. 286–297, 2000.

[27] J. W. Crandall, M. A. Goodrich, D. R. Olsen, and C. W. Nielsen, "Validating human–robot interaction schemes in multitasking environments," *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, vol. 35, no. 4, pp. 438–449, 2005.

[28] M. L. Cummings and P. J. Mitchell, "Predicting controller capacity in supervisory control of multiple UAVs," *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, vol. 38, no. 2, pp. 451–460, 2008.

[29] K. J. K. Feng, D. W. McDonald, and A. X. Zhang, "Levels of autonomy for AI agents." arXiv preprint arXiv:2506.12469, 2025.

[30] D. McCreary, "AI for EE research: From researcher to orchestrator." Interactive intelligent textbook, `https://dmccreary.github.io/ai-for-ee-research/`, 2026. Source repository: `https://github.com/dmccreary/ai-for-ee-research`. Accessed 2026-10-22.

[31] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large language model society." arXiv preprint arXiv:2303.17760, 2023.

[32] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face." arXiv preprint arXiv:2303.17580, 2023.

[33] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2609–2634, Association for Computational Linguistics, 2023.

[34] E. Schluntz and B. Zhang, "Building effective agents." Anthropic Engineering, `https://www.anthropic.com/engineering/building-effective-agents`, Dec. 2024. Accessed 2026-10-22.

[35] J. Hadfield, B. Zhang, K. Lien, F. Scholz, J. Fox, and D. Ford, "How we built our multi-agent research system." Anthropic Engineering, `https://www.anthropic.com/engineering/multi-agent-research-system`, June 2025. Accessed 2026-10-22.

[36] Agentic AI Foundation, "AGENTS.md: A simple, open format for guiding coding agents." `https://agents.md/`, 2025. Accessed 2026-10-22.

[37] Agent Skills, "Agent skills specification." `https://agentskills.io/specification`, 2025. Accessed 2026-10-22.

[38] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?." arXiv preprint arXiv:2310.06770, 2023.

[39] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, no. 7992, pp. 570–578, 2023.

[40] A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller, "Augmenting large language models with chemistry tools," *Nature Machine Intelligence*, vol. 6, no. 5, pp. 525–535, 2024.

[41] J. Gottweis, W.-H. Weng, A. Daryin, T. Tu, P. Sirkovic, A. Myaskovsky, G. Glowaty, F. Weissenberger, A. Orlandi, D. Popovici, *et al.*, "Accelerating scientific discovery with Co-Scientist." arXiv preprint arXiv:2502.18864, 2025.

[42] L. Messeri and M. J. Crockett, "Artificial intelligence and illusions of understanding in scientific research," *Nature*, vol. 627, no. 8002, pp. 49–58, 2024.

[43] M. Mitchell, A. Ghosh, A. S. Luccioni, and G. Pistilli, "Fully autonomous AI agents should not be developed." arXiv preprint arXiv:2502.02649, 2025.

[44] E. Horvitz, "Principles of mixed-initiative user interfaces," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '99)*, pp. 159–166, ACM Press, 1999.

[45] S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz, "Guidelines for human-AI interaction," in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pp. 1–13, ACM, 2019.

[46] B. Shneiderman, "Human-centered artificial intelligence: Reliable, safe & trustworthy," *International Journal of Human–Computer Interaction*, vol. 36, no. 6, pp. 495–504, 2020.

[47] A. Chan, C. Ezell, M. Kaufmann, K. Wei, L. Hammond, H. Bradley, E. Bluemke, N. Rajkumar, D. Krueger, N. Kolt, L. Heim, and M. Anderljung, "Visibility into AI agents," in *The 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT '24)*, pp. 958–973, ACM, 2024.

[48] W. Ketter, A. Batchu, G. Berosik, and D. McCreary, "A semantic web architecture for advocate agents to determine preferences and facilitate decision making," in *Proceedings of the 10th International Conference on Electronic Commerce (ICEC '08)*, pp. 1–10, ACM, 2008.

[49] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Human Factors*, vol. 52, no. 3, pp. 381–410, 2010.

[50] K. Goddard, A. Roudsari, and J. C. Wyatt, "Automation bias: a systematic review of frequency, effect mediators, and mitigators," *Journal of the American Medical Informatics Association*, vol. 19, no. 1, pp. 121–127, 2012.

[51] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[52] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, pp. 79–90, ACM, 2023.

[53] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena." arXiv preprint arXiv:2306.05685, 2023.

[54] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, "Ten simple rules for reproducible computational research," *PLoS Computational Biology*, vol. 9, no. 10, p. e1003285, 2013.

[55] G. Wilson, D. A. Aruliah, C. T. Brown, N. P. Chue Hong, M. Davis, R. T. Guy, S. H. D. Haddock, K. D. Huff, I. M. Mitchell, M. D. Plumbley, B. Waugh, E. P. White, and P. Wilson, "Best practices for scientific computing," *PLoS Biology*, vol. 12, no. 1, p. e1001745, 2014.

[56] G. Wilson, J. Bryan, K. Cranston, J. Kitzes, L. Nederbragt, and T. K. Teal, "Good enough practices in scientific computing," *PLOS Computational Biology*, vol. 13, no. 6, p. e1005510, 2017.

[57] W. L. Oberkampf and C. J. Roy, *Verification and Validation in Scientific Computing*. Cambridge: Cambridge University Press, 2010.

[58] J. E. Kelley, Jr. and M. R. Walker, "Critical-path planning and scheduling," in *Papers Presented at the December 1–3, 1959, Eastern Joint IRE-AIEE-ACM Computer Conference*, pp. 160–173, ACM Press, 1959.

[59] F. P. Brooks, Jr., *The Mythical Man-Month: Essays on Software Engineering*. Reading, MA: Addison-Wesley, anniversary ed., 1995.

[60] R. K. Yin, *Case Study Research and Applications: Design and Methods*. Thousand Oaks, CA: SAGE Publications, sixth ed., 2018.

[61] V. Lockhart, D. McCreary, and T. A. Peterson, "MicroSims: A framework for AI-generated, scalable educational simulations with universal embedding and adaptive learning support." arXiv preprint arXiv:2511.19864, 2025.

[62] IEEE, "IEEE standard for terminology and test methods for analog-to-digital converters." IEEE Std 1241-2010, 2011.

[63] D. McCreary, "Integrating personal knowledge graphs into the enterprise," in *Personal Knowledge Graphs (PKGs): Methodology, Tools and Applications*, ch. 14, pp. 307–325, Institution of Engineering and Technology, 2023.